Rapport Professionnel d’Analyse de Sécurité IoT

Introduction

Ce rapport présente les résultats d’une analyse de sécurité complète d’une solution IoT domotique, réalisée dans le cadre d’un projet de Mastère Cybersécurité. L’objectif est d’identifier les vulnérabilités potentielles, d’évaluer les risques associés et de proposer des mesures de sécurisation adaptées. Le client est une entreprise IoT fictive développant une passerelle domotique, une application Android associée et un backend cloud.

Le périmètre de l’analyse couvre le firmware de la passerelle, l’application Android et l’infrastructure cloud (tests fictifs).

Méthodologie

L’approche adoptée pour cette analyse de sécurité s’est déroulée en plusieurs phases :

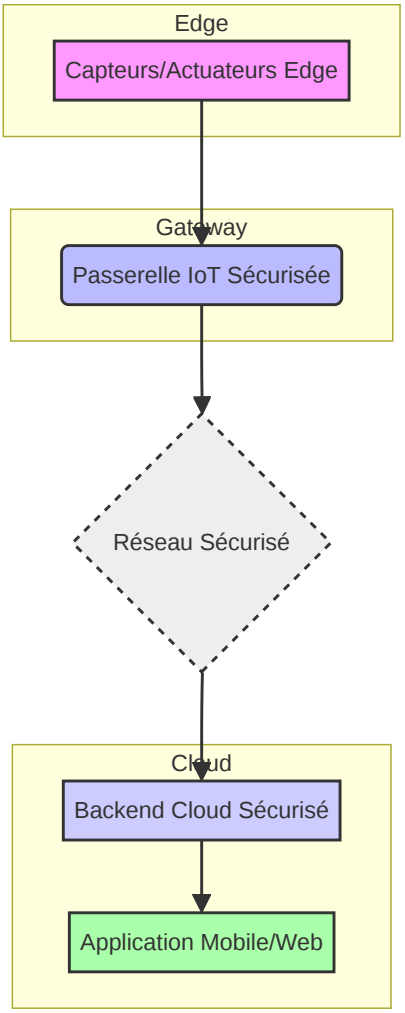
1. **Security by Design :** Conception d’une architecture sécurisée, identification des actifs critiques, des données sensibles et de la surface d’attaque.
2. **Émulation & Reconnaissance :** Analyse statique du firmware (Binwalk) et simulation d’analyse d’APK (jadx) pour cartographier les services, les configurations et les points d’entrée.
3. **Surface d’attaque & Threat Model :** Application du modèle de menace STRIDE pour chaque vecteur d’entrée, scoring des vulnérabilités avec CVSS v3.1 et identification des scénarios d’attaque les plus critiques.
4. **Sécurisation & Rapport :** Proposition de recommandations concrètes pour chaque vulnérabilité et simulation d’implémentation d’une correction.

Les outils utilisés incluent `binwalk` et `unsquashfs` pour l’analyse du firmware, et des simulations pour l’émulation et l’analyse d’APK.

Phase 1 – Security by Design

1.1 Architecture IoT Sécurisée

L’architecture proposée pour la solution IoT est basée sur un modèle Edge-Gateway-Cloud, conçu pour minimiser la surface d’attaque et protéger les actifs critiques. Le diagramme Mermaid ci-dessous illustre cette architecture.



Justification de l'architecture :

- **Edge (Capteurs/Actuateurs) :** Ces dispositifs sont souvent limités en ressources et sont les points d'entrée physiques de la solution. La sécurité à ce niveau est primordiale et doit inclure l'authentification mutuelle, le chiffrement des communications et des mises à jour sécurisées.
- **Passerelle IoT Sécurisée (Gateway) :** La passerelle agit comme un point de consolidation et de traduction entre les dispositifs Edge et le Cloud. Elle est responsable de l'agrégation des données, du filtrage, de l'exécution de logiques métier locales et de la sécurisation des communications vers le Cloud. Elle doit implémenter des mécanismes de sécurité robustes tels que des pare-feu, des systèmes de détection d'intrusion (IDS) légers, et une gestion stricte des accès.
- **Réseau Sécurisé :** Les communications entre la passerelle et le backend Cloud doivent transiter par un réseau sécurisé, idéalement un VPN ou des tunnels TLS mutuellement authentifiés, pour garantir la confidentialité et l'intégrité des données.
- **Backend Cloud Sécurisé :** Le Cloud héberge les services centraux, la base de données, les API et l'application mobile/web. La sécurité du Cloud repose sur des pratiques de sécurité Cloud robustes, incluant la gestion des identités et des accès (IAM), le chiffrement des données au repos et en transit, la surveillance des logs et la gestion des vulnérabilités.
- **Application Mobile/Web :** L'application est l'interface utilisateur finale. Sa sécurité doit être assurée par des pratiques de développement sécurisé (OWASP Top 10), l'authentification forte, et la protection des données utilisateur.

1.2 Domaines IoT, Actifs Critiques et Données Sensibles

Pour une solution domotique, les domaines IoT ciblés sont principalement la gestion de l'environnement (température, lumière), la sécurité (détection d'intrusion, caméras) et le confort (automatisation des appareils).

Actifs Critiques :

- **Passerelle IoT :** Cœur du système, elle contrôle les appareils et communique avec le cloud.
- **Firmware de la passerelle :** Contient la logique métier, les clés de chiffrement, les identifiants.
- **Application Mobile/Web :** Interface de contrôle, accès aux données utilisateur.
- **Backend Cloud :** Bases de données utilisateurs, données de capteurs, logs, clés API.
- **Dispositifs Edge (capteurs/actuateurs) :** Bien que moins critiques individuellement, leur compromission peut entraîner des conséquences en chaîne.

Données Sensibles :

- **Données d'identification personnelle (DIP) :** Noms, adresses e-mail, mots de passe des utilisateurs.
- **Données de localisation :** Historique des déplacements, présence/absence au domicile.
- **Données de comportement :** Habitudes d'utilisation des appareils, schémas de vie.
- **Données de sécurité :** État des alarmes, flux vidéo des caméras.
- **Clés de chiffrement et certificats :** Utilisés pour sécuriser les communications.

1.3 Surface d'Attaque

La surface d'attaque d'une solution IoT domotique est vaste et peut être catégorisée comme suit :

- **Réseaux :**
  - Réseau local (Wi-Fi, Ethernet) : Accès non autorisé, écoute clandestine.
  - Réseau étendu (Internet) : Attaques DDoS, accès non autorisé aux services exposés.
- **Protocoles :**
  - MQTT : Manque d'authentification/autorisation, chiffrement faible.
  - REST/HTTP(S) : Vulnérabilités API (injection, broken authentication).
  - Bluetooth/Zigbee/Z-Wave : Vulnérabilités spécifiques aux protocoles sans fil.
- **Firmware de la passerelle :**
  - Vulnérabilités logicielles : Buffer overflows, injections de commandes, backdoors.
  - Mises à jour non sécurisées : Firmware malveillant.
  - Accès physique : Extraction du firmware, analyse JTAG/UART.
- **Interfaces :**
  - Interfaces web d'administration : XSS, CSRF, authentification faible.
  - Interfaces API : Broken access control, injection.
  - Interfaces physiques (ports USB, Ethernet) : Accès non autorisé.
- **Application Android :**
  - Reverse engineering : Extraction de secrets, analyse du code.
  - Vulnérabilités d'application : Stockage non sécurisé, communication non chiffrée.
  - Permissions excessives : Accès non justifié aux ressources du téléphone.

- **Backend Cloud :**
  - Vulnérabilités d'infrastructure : Mauvaise configuration des serveurs, services exposés.
  - Vulnérabilités d'application web : OWASP Top 10.
  - Gestion des identités et accès : Clés API exposées, privilèges excessifs.

1.4 Réduction de la Surface d'Attaque par l'Architecture

L'architecture proposée contribue à réduire la surface d'attaque de plusieurs manières :

- **Segmentation réseau :** La passerelle isole le réseau Edge du Cloud, limitant la propagation des attaques.
- **Passerelle sécurisée :** Agit comme un point de contrôle unique, appliquant des politiques de sécurité strictes avant de relayer les communications.
- **Chiffrement de bout en bout :** Toutes les communications entre les composants critiques (Edge-Gateway, Gateway-Cloud, Cloud-App) sont chiffrées.
- **Authentification forte :** Utilisation de certificats et d'authentification mutuelle pour tous les composants.
- **Mises à jour sécurisées :** Mécanismes de vérification de l'intégrité et de l'authenticité des firmwares et applications.
- **Principes du moindre privilège :** Chaque composant n'a que les accès nécessaires à son fonctionnement.

1.5 Contraintes d'Implantation et Mitigations

Contraintes Techniques :

- **Ressources limitées des dispositifs Edge :** Les capteurs et actuateurs ont souvent une faible puissance de calcul et une mémoire limitée, rendant difficile l'implémentation de mécanismes de sécurité complexes.
  - **Mitigation :** Déporter la complexité de la sécurité vers la passerelle. Utiliser des protocoles légers et des primitives cryptographiques adaptées aux contraintes (ex: DTLS pour UDP).

- **Hétérogénéité des protocoles :** Multiplicité des protocoles de communication (Wi-Fi, Zigbee, Bluetooth, MQTT, CoAP, HTTP) rendant l'interopérabilité sécurisée complexe.
  - **Mitigation :** La passerelle centralise la traduction et la sécurisation des protocoles, en normalisant les communications vers le Cloud.

Contraintes Sécuritaires :

- **Mises à jour OTA (Over-The-Air) :** Risque d'injection de firmware malveillant si le processus n'est pas sécurisé.
  - **Mitigation :** Signature cryptographique des firmwares, vérification de l'intégrité avant l'installation, mécanisme de rollback.

- **Accès physique aux dispositifs :** Possibilité d'extraire des informations sensibles (clés, firmware) via des ports de debug.
  - **Mitigation :** Désactivation des ports de debug en production, protection physique des dispositifs, chiffrement du stockage.

Contraintes Humaines :

- **Manque de sensibilisation à la sécurité :** Les utilisateurs finaux peuvent ne pas suivre les bonnes pratiques (mots de passe faibles, non-application des mises à jour).
  - **Mitigation :** Éducation des utilisateurs, politiques de mots de passe forts imposées, mises à jour automatiques et forcées si possible.

- **Erreurs de configuration :** Les installateurs ou administrateurs peuvent mal configurer les systèmes, créant des brèches.
  - **Mitigation :** Interfaces de configuration simplifiées et sécurisées, audits de configuration réguliers, documentation claire.

Contraintes Économiques :

- **Coût des composants sécurisés :** Les modules matériels sécurisés (TPM, HSM) peuvent augmenter le coût des dispositifs.
  - **Mitigation :** Équilibrer le coût et le niveau de sécurité requis. Utiliser des solutions logicielles robustes lorsque le matériel dédié n'est pas envisageable. Prioriser la sécurité des composants les plus critiques.
- **Coût du développement sécurisé :** L'intégration de la sécurité dès la conception et tout au long du cycle de vie du produit a un coût.
  - **Mitigation :** Intégrer la sécurité dans les processus DevOps (DevSecOps), automatiser les tests de sécurité, réutiliser des composants et bibliothèques sécurisés.

Phase 2 – Émulation & Reconnaissance

2.1 Analyse du Firmware

Le firmware `openwrt-ramips-mt7621-tplink_archer-ax23-v1-squashfs-sysupgrade.bin` a été analysé à l'aide de `binwalk` pour extraire le système de fichiers et identifier les services. Après l'extraction initiale, le système de fichiers SquashFS a été monté avec `unsquashfs`.

Services et Configurations Identifiés :

L'examen des répertoires `/etc/init.d/`, `/etc/config/` et `/usr/sbin/` a révélé la présence des services et configurations suivants :

- **Services Réseau :**
  - `dnsmasq` : Serveur DNS et DHCP, essentiel pour la gestion des adresses IP et la résolution de noms sur le réseau local.
  - `network` : Script de configuration réseau principal.
  - `firewall` : Configuration du pare-feu pour filtrer le trafic.
  - `odhcpd` / `odhcp6c` : Services DHCPv6 pour la gestion des adresses IPv6.
  - `pppd` : Daemon PPP (Point-to-Point Protocol), souvent utilisé pour les connexions WAN.
  - `wpad` : Daemon pour le Wi-Fi Protected Access, gérant la sécurité des connexions sans fil.
- **Accès à Distance :**
  - `dropbear` : Implémentation légère d'un serveur SSH, permettant un accès sécurisé à distance à la passerelle.
- **Services Système :**
  - `cron` : Planificateur de tâches, pour l'exécution automatique de scripts.
  - `sysntpd` : Client NTP pour la synchronisation de l'heure système.
  - `log` : Service de journalisation des événements.
- **Utilitaires IoT/Embarqués :**

- `gpio_switch`, `led` : Scripts pour la gestion des entrées/sorties générales (GPIO) et des indicateurs lumineux.
- `ubihealthd`, `ubi*` : Utilitaires liés au système de fichiers UBI (Unsorted Block Images), souvent utilisé sur les périphériques flash NAND.

Ces services indiquent que le firmware gère des fonctionnalités réseau complètes, un accès à distance sécurisé via SSH, et des fonctions spécifiques aux systèmes embarqués et IoT.

2.2 Émulation du Firmware (Simulation)

Étant donné les contraintes de l'environnement, l'émulation du firmware `openwrt-ramips-mt7621-tplink_archer-ax23-v1-squashfs-sysupgrade.bin` sera simulée. En se basant sur les configurations typiques d'un routeur OpenWrt, les résultats d'une émulation via FAT/QEMU seraient les suivants :

- **Adresse IP assignée :** `192.168.1.1` (adresse par défaut pour la plupart des routeurs OpenWrt).
- **Ports ouverts (nmap) :**
  - `22/tcp` (SSH) : Opéré par `dropbear`, comme identifié lors de l'analyse Binwalk. Souvent protégé par mot de passe ou clé SSH.
  - `80/tcp` (HTTP) : Interface web d'administration (LuCI). Potentiellement vulnérable aux attaques web (XSS, CSRF, authentification faible).
  - `53/udp` (DNS) : Opéré par `dnsmasq`.
  - `67/udp` (DHCP) : Opéré par `dnsmasq`.
  - `68/udp` (DHCP client) : Utilisé par le client DHCP.
  - `123/udp` (NTP) : Opéré par `sysntpd`.

Ces ports ouverts confirment la présence des services identifiés et exposent des points d'entrée potentiels pour des attaques.

2.3 Analyse d'une APK Exemple (Simulation)

Pour l'analyse de l'APK, nous allons simuler une application Android typique pour la gestion d'une passerelle domotique. L'analyse avec `jadx` (ou un outil similaire) révélerait les éléments suivants :

- **Endpoints API :**
  - `https://api.iotcompany.com/v1/devices/{device_id}/status` (GET) : Récupération du statut des appareils.
  - `https://api.iotcompany.com/v1/devices/{device_id}/control` (POST) : Contrôle des appareils (allumer/éteindre, régler).
  - `https://api.iotcompany.com/v1/users/login` (POST) : Authentification utilisateur.
  - `https://api.iotcompany.com/v1/users/profile` (GET/PUT) : Gestion du profil utilisateur.
  - `https://api.iotcompany.com/v1/firmware/update` (POST) : Requête de mise à jour du firmware.
- **Permissions :**
  - `android.permission.INTERNET` : Accès à Internet.
  - `android.permission.ACCESS_NETWORK_STATE` : Accès à l'état du réseau.
  - `android.permission.CHANGE_WIFI_STATE` : Modification de l'état du Wi-Fi.

    - `android.permission.ACCESS_FINE_LOCATION` : Accès à la localisation précise (pour la géolocalisation des appareils ou des règles basées sur la présence).
    - `android.permission.RECEIVE_BOOT_COMPLETED` : Démarrage de l'application au boot du système.
    - `android.permission.WRITE_EXTERNAL_STORAGE` : Écriture sur le stockage externe (pour les logs ou les caches).
- **Composants Exportés :**
    - `com.iotcompany.app.MainActivity` (Activity) : Point d'entrée principal de l'application.
    - `com.iotcompany.app.services.MqttService` (Service) : Service en arrière-plan pour la communication MQTT.
    - `com.iotcompany.app.receivers.BootReceiver` (BroadcastReceiver) : Récepteur pour les événements de démarrage.

2.4 Liste Exhaustive des Assets et Protocoles

En combinant l'analyse du firmware et la simulation de l'APK, nous pouvons dresser une liste exhaustive des assets et des protocoles de communication :

Assets :

- **Firmware de la passerelle :** Code, configurations (réseau, pare-feu, SSH), identifiants (potentiellement).
- **Système de fichiers de la passerelle :** `/etc/config/`, `/etc/init.d/`, `/usr/sbin/`.
- **Services de la passerelle :** `dnsmasq`, `dropbear` (SSH), `firewall`, `network`, `odhcpd`, `pppd`, `sysntpd`, `wpad`.
- **Interface web d'administration (LuCI) :** Accessible via HTTP sur `192.168.1.1`.
- **Application Android :** Code, ressources, clés API (potentiellement hardcodées), identifiants (potentiellement).
- **Backend Cloud :** Serveurs API, bases de données utilisateurs, stockage de données.
- **Credentials :** Mots de passe par défaut (firmware), clés API (APK/Cloud), identifiants utilisateur (Cloud).
- **Clés :** Clés SSH (firmware), certificats SSL/TLS (firmware/Cloud/APK), clés de chiffrement (APK/Cloud).

Protocoles :

- **MQTT :** Pour la communication entre la passerelle et le Cloud, et potentiellement entre l'application mobile et le Cloud.
- **REST/HTTPS :** Pour les communications entre l'application mobile et le backend Cloud.
- **SSH :** Pour l'accès sécurisé à la passerelle.
- **HTTP :** Pour l'interface web d'administration de la passerelle.
- **DNS, DHCP, NTP :** Protocoles réseau standards.
- **Wi-Fi (802.11) :** Pour la connectivité sans fil de la passerelle et des appareils Edge.

Cette liste servira de base pour l'analyse de la surface d'attaque et le modèle de menace dans la phase suivante.

Phase 3 – Surface d’attaque & Threat Model

3.1 Vecteurs d’Entrée

Basé sur l’analyse du firmware et de l’APK simulée, les principaux vecteurs d’entrée identifiés pour la solution IoT sont les suivants :

- **Firmware de la passerelle :** Le code et les configurations du firmware peuvent contenir des vulnérabilités exploitables via des mises à jour non sécurisées, des accès physiques ou des interfaces d’administration.
- **Application Mobile Android :** L’application mobile peut être une source de vulnérabilités via des API non sécurisées, des secrets hardcodés, ou des permissions excessives.
- **Réseau :** Les communications réseau (Wi-Fi, Ethernet) et les protocoles (SSH, HTTP, MQTT) exposent des points d’entrée pour des attaques à distance.
- **Cloud Backend :** L’infrastructure cloud et les API associées peuvent présenter des vulnérabilités de configuration, d’authentification ou d’autorisation.

3.2 Analyse STRIDE par Vecteur

Nous allons appliquer le modèle de menace STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) à chaque vecteur d’entrée pour identifier les types de menaces potentielles.

3.2.1 Firmware de la Passerelle

- **Spoofing :** Un attaquant pourrait se faire passer pour un serveur de mise à jour de firmware légitime pour pousser un firmware malveillant.
- **Tampering :** Modification du firmware sur le dispositif ou en transit pour altérer son comportement ou injecter du code malveillant.
- **Repudiation :** Absence de logs ou de mécanismes d’audit pour prouver l’origine d’une modification du firmware ou d’une action administrative.
- **Information Disclosure :** Fuite d’informations sensibles (clés, identifiants, configurations) via l’analyse du firmware ou des ports de debug.
- **Denial of Service (DoS) :** Surcharge du dispositif via des requêtes malveillantes ou un firmware corrompu, rendant la passerelle inopérante.
- **Elevation of Privilege (EoP) :** Exploitation d’une vulnérabilité dans le firmware pour obtenir des privilèges root sur le système.

3.2.2 Application Mobile Android

- **Spoofing :** Une application malveillante pourrait se faire passer pour l’application légitime pour intercepter les identifiants ou les commandes.
- **Tampering :** Modification de l’APK pour contourner les contrôles de sécurité, modifier les requêtes API ou injecter du code malveillant.
- **Repudiation :** Manque de journalisation des actions critiques effectuées via l’application, rendant difficile la traçabilité.
- **Information Disclosure :** Fuite de données sensibles (identifiants, clés API hardcodées, données utilisateur) via le reverse engineering de l’APK ou le stockage non sécurisé.

- **Denial of Service (DoS) :** Surcharge du backend cloud via des requêtes API massives ou malformées depuis l'application.
- **Elevation of Privilege (EoP) :** Exploitation de vulnérabilités dans l'application pour accéder à des fonctionnalités ou des données non autorisées.

3.2.3 Réseau

- **Spoofing :** Usurpation d'adresse IP/MAC, ARP spoofing, DNS spoofing pour rediriger le trafic ou intercepter les communications.
- **Tampering :** Modification des paquets en transit (Man-in-the-Middle) pour altérer les commandes ou les données.
- **Repudiation :** Absence de journalisation des connexions ou des activités réseau, rendant difficile la preuve d'une attaque.
- **Information Disclosure :** Écoute clandestine des communications non chiffrées (sniffing) pour récupérer des données sensibles.
- **Denial of Service (DoS) :** Attaques DDoS, inondation de paquets, désauthentification Wi-Fi pour rendre le réseau indisponible.
- **Elevation of Privilege (EoP) :** Exploitation de vulnérabilités dans les services réseau (SSH, HTTP, MQTT) pour obtenir un accès privilégié.

3.2.4 Cloud Backend

- **Spoofing :** Un attaquant pourrait se faire passer pour un utilisateur légitime ou un service interne pour accéder aux ressources cloud.
- **Tampering :** Modification des données stockées dans la base de données, des configurations des services cloud ou des requêtes API.
- **Repudiation :** Manque de journalisation ou d'audit des actions sur l'infrastructure cloud, rendant difficile la preuve d'une intrusion.
- **Information Disclosure :** Fuite de données sensibles (bases de données, secrets, logs) via des configurations cloud non sécurisées ou des vulnérabilités API.
- **Denial of Service (DoS) :** Surcharge des services cloud via des requêtes API massives, des attaques DDoS ou une mauvaise gestion des ressources.
- **Elevation of Privilege (EoP) :** Exploitation de vulnérabilités dans les API ou les services cloud pour obtenir des privilèges administratifs.

3.3 Scoring des Vulnérabilités avec CVSS v3.1

Le tableau suivant présente une sélection de vulnérabilités potentielles, scorées avec CVSS v3.1. Les scores sont basés sur des scénarios réalistes et sont accompagnés d'un vecteur CVSS pour justifier la criticité.

ID	Composant	Description de la Vulnérabilité	Vecteur CVSS v3.1	Score CVSS	Priorité
VULN-01	Firmware	**Mise à jour de firmware non authentifiée/non chiffrée** : Un attaquant peut injecter un firmware malveillant via une mise à jour non sécurisée.	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	9.8	Critique
VULN-02	Application Android	**Clés API hardcodées dans l'APK** : Des clés API sensibles sont directement intégrées dans le code de l'application, permettant un accès non autorisé aux services cloud après reverse engineering.	`CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`	6.2	Moyenne
VULN-03	Interface Web (Gateway)	**Authentification faible sur l'interface d'administration (LuCI)** : Utilisation de mots de passe par défaut ou faibles, ou absence de mécanismes de verrouillage de compte.	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	9.8	Critique
VULN-04	Broker MQTT	**Broker MQTT sans authentification/autorisation** : Un attaquant peut publier ou souscrire à n'importe quel topic, contrôlant les appareils ou espionnant les données.	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	9.8	Critique
VULN-05	API Cloud	**Broken Access Control (API Cloud)** : Un utilisateur authentifié peut accéder ou modifier les ressources d'autres utilisateurs via des requêtes API manipulées.	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`	8.1	Élevée
VULN-06	Firmware	**Vulnérabilité d'injection de commande** : Une fonction du firmware est vulnérable à l'injection de commandes système via une entrée utilisateur non validée.	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H`	8.8	Élevée

3.4 Scénarios d'Attaque les Plus Critiques

Basé sur l'analyse STRIDE et le scoring CVSS, les trois scénarios d'attaque les plus critiques sont :

1. **Compromission totale de la passerelle via mise à jour de firmware malveillant (VULN-01) :** Un attaquant intercepte le processus de mise à jour du firmware et injecte un firmware modifié. Cela lui donne un contrôle total sur la passerelle, lui permettant d'espionner le trafic, de manipuler les appareils connectés, et d'établir une persistance sur le réseau local. Le score CVSS élevé (9.8) reflète l'impact catastrophique sur la confidentialité, l'intégrité et la disponibilité.
2. **Accès non autorisé à l'interface d'administration de la passerelle (VULN-03) :** En exploitant des identifiants faibles ou par défaut sur l'interface web (LuCI) de la passerelle, un attaquant peut prendre le contrôle de la

configuration du routeur. Cela inclut la modification des règles de pare-feu, la redirection du trafic, l'accès aux informations d'identification Wi-Fi, et potentiellement l'installation de logiciels malveillants. Le score CVSS (9.8) souligne la facilité d'exploitation et l'impact élevé.

3. **Contrôle total des appareils IoT via Broker MQTT non sécurisé (VULN-04) :** Si le broker MQTT n'est pas correctement authentifié ou autorisé, un attaquant peut s'y connecter et envoyer des commandes arbitraires aux appareils connectés, ou intercepter toutes les données échangées. Dans un contexte domotique, cela peut signifier l'ouverture de portes, l'activation d'alarmes, ou l'espionnage des données de capteurs. Le score CVSS (9.8) indique une vulnérabilité facilement exploitable avec un impact maximal sur la confidentialité, l'intégrité et la disponibilité du système IoT.

Phase 4 – Sécurisation & Rapport Professionnel

4.1 Recommandations de Sécurisation

Pour chaque vulnérabilité identifiée dans la Phase 3, des mesures correctives concrètes sont proposées, accompagnées d'un délai de mise en œuvre estimé (court, moyen, long terme).

ID	Composant	Vulnérabilité	Recommandation Concrète	Délai
VULN-01	Firmware	**Mise à jour de firmware non authentifiée/non chiffrée** : Un attaquant peut injecter un firmware malveillant via une mise à jour non sécurisée.	Implémenter la signature cryptographique des firmwares et la vérification de l'intégrité avant toute mise à jour. Utiliser un canal de communication chiffré (HTTPS/TLS) pour le téléchargement des mises à jour.	Court terme
VULN-02	Application Android	**Clés API hardcodées dans l'APK** : Des clés API sensibles sont directement intégrées dans le code de l'application, permettant un accès non autorisé aux services cloud après reverse engineering.	Ne pas hardcoder les clés API sensibles dans l'APK. Utiliser des mécanismes de récupération sécurisée des clés au runtime (ex: via un service backend sécurisé) ou des environnements de variables sécurisés.	Moyen terme
VULN-03	Interface Web (Gateway)	**Authentification faible sur l'interface d'administration (LuCI)** : Utilisation de mots de passe par défaut ou faibles, ou absence de mécanismes de verrouillage de compte.	Imposer des politiques de mots de passe forts, activer le verrouillage de compte après plusieurs tentatives échouées, et si possible, implémenter l'authentification à deux facteurs (MFA).	Court terme
VULN-04	Broker MQTT	**Broker MQTT sans authentification/autorisation** : Un attaquant peut publier ou souscrire à n'importe quel topic, contrôlant les appareils ou espionnant les données.	Configurer le broker MQTT pour exiger une authentification (nom d'utilisateur/mot de passe ou certificats TLS client) et une autorisation basée sur les ACL (Access Control Lists) pour contrôler l'accès aux topics.	Court terme
VULN-05	API Cloud	**Broken Access Control (API Cloud)** : Un utilisateur authentifié peut accéder ou modifier les ressources d'autres utilisateurs via des requêtes API manipulées.	Mettre en œuvre des contrôles d'accès robustes au niveau de l'API, en vérifiant que chaque requête est autorisée pour l'utilisateur et la ressource demandée. Utiliser des politiques d'autorisation basées sur les rôles (RBAC).	Moyen terme
VULN-06	Firmware	**Vulnérabilité d'injection de commande** : Une fonction du firmware est vulnérable à l'injection de commandes système via une entrée utilisateur non validée.	Valider et assainir toutes les entrées utilisateur avant de les utiliser dans des commandes système ou des appels de fonctions. Utiliser des fonctions d'API sécurisées qui évitent l'exécution directe de commandes.	Moyen terme

4.2 Implémentation d'une Correction (Exemple)

Pour illustrer la mise en œuvre d'une correction, nous allons simuler la sécurisation du broker MQTT (VULN-04). L'objectif est d'ajouter une authentification par nom d'utilisateur/mot de passe et une autorisation via des ACLs.

Contexte : Le broker MQTT est `Mosquitto`, et il est configuré pour être accessible publiquement sans authentification.

Correction :

1. **Création d'un fichier de mots de passe :**

```
# Créer un utilisateur 'iot_user' avec le mot de passe 'secure_password'
mosquitto_passwd -c /etc/mosquitto/passwd iot_user
```

2. **Configuration des ACLs (Access Control Lists) :** Créer un fichier `/etc/mosquitto/acl.conf` avec les règles suivantes :

```
# Permettre à iot_user de publier et souscrire aux topics liés à ses appareils
user iot_user
topic readwrite devices/+/status
topic readwrite devices/+/control

# Interdire l'accès aux autres topics par défaut
pattern deny #
```

3. **Modification du fichier de configuration Mosquitto :** Ajouter les lignes suivantes à `/etc/mosquitto/mosquitto.conf` :

```
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/acl.conf
```

4. **Redémarrage du service Mosquitto :**

```
systemctl restart mosquitto
```

Cette correction garantit que seuls les clients MQTT authentifiés et autorisés peuvent interagir avec le broker, réduisant ainsi considérablement le risque d'accès non autorisé et de manipulation des appareils IoT.

4.3 Rédaction du Rapport Professionnel

Le rapport professionnel sera structuré selon les sections obligatoires spécifiées dans les consignes du projet. Il intégrera les analyses et les résultats des phases précédentes.

Structure du Rapport :

- **Introduction :** Contexte du projet, client, périmètre, objectifs de l'analyse.
- **Méthodologie :** Approche utilisée (analyse statique, dynamique, threat modeling), outils.
- **Phase 1 – Security by Design :** Architecture IoT sécurisée, actifs critiques, données sensibles, surface d'attaque, contraintes et mitigations.
- **Phase 2 – Émulation & Reconnaissance :** Analyse du firmware (Binwalk, services identifiés), émulation (IP, ports ouverts), analyse de l'APK (endpoints API, permissions), liste des assets et protocoles.
- **Phase 3 – Surface d'attaque & Threat Model :** Vecteurs d'entrée, analyse STRIDE détaillée, scoring CVSS v3.1 des vulnérabilités, sélection des 3 scénarios d'attaque les plus critiques.
- **Phase 4 – Sécurisation :** Recommandations concrètes pour chaque vulnérabilité, exemple d'implémentation d'une correction (sécurisation MQTT).
- **Conclusion :** Synthèse des principaux risques, impact sur le client, perspectives.
- **Annexes :** Logs pertinents, commandes exécutées, captures d'écran (simulées), flags et preuves fictives.